Generative Deep Learning (708.088) SS26
Assignment 1

Autoregressive Models & Latent Variable Models

Dr. Ozan Özdenizci
Institute of Machine Learning and Neural Computation, TU Graz
oezdenizci@tugraz.at

Points to achieve:	12 pts
Assignment Issued:	19.03.2026 09:00 AM
Deadline:	16.04.2026 09:00 AM (**no extensions**)
Hand-in procedure:	Individual submission (no groups allowed for this assignment). **The report (.pdf)** and **the code skeleton solution as .ipynb**. **Do not upload a folder. Do not zip the files.**
Assignment interviews:	You might be invited for an assignment interview.
Plagiarism:	If detected, 0 points from the assignment. Any use of LLMs or other AI-assisted tools must be clearly disclosed. Undisclosed use may be considered plagiarism.

- run it on google colab so it doesn't take so long

- what did he mean with watermarks

General Remarks

- For all the tasks below, create the appropriate code and discuss your experimentation and reasoning process, findings and choices in your report.
- All results (i.e., plots, results of computations) should be included in your PDF report.
- The submitted code as a Jupyter Notebook file should be able to reproduce your results.
- For all tasks, implement the required code and answer the associated questions in the report.
- Make sure all of your plots have labeled axes and are clearly described in your report.
- You are *only* allowed to add code to the respective `TODO` sections.

  -> you can change stuff if you explain it
- You should start experiment with the provided hyper-parameters by default. You are also free to adapt these hyper-parameters later, by justifying the reason why.
- Commented out code will not be executed or graded.

Assignment 1: Training a VQ-VAE with an Autoregressive Prior

In this assignment, you will first implement a Vector-Quantized Variational Autoencoder (VQ-VAE), and then train an autoregressive prior over its discrete latent space using the MNIST dataset.

The model will be trained in two stages: (1) You will first train the VQ-VAE (encoder, vector quantizer with the codebook, and decoder), to reconstruct inputs through a discrete bottleneck. (2) After the VQ-VAE is trained, you will freeze the encoder and extract the latent code sequences, and use these sequences to train an autoregressive model over the discrete latent codes with 1D causal convolutions, to capture dependencies between latent positions.

-> same code as in first jupyter notebook

- you learn which token follows which to generate a certain digit

Finally, you will use the learned prior to generate new latent sequences and decode them into images, comparing this approach to generation without a learned prior. The goal is to understand how discrete latent representations and autoregressive models can be combined for generative modeling.

-> i.e. if you randomly pick codebook vectors

Task details:

a) (2 pts) : Complete the `TODO` of the `VectorQuantizer` class forward pass under *"Part (a): VQ-VAE architecture"*. Provide the code snippet of your implementation. Explain how you obtained the list of codebook `indices`, the quantized output $\mathbf{z}_q$, and the two terms contributing to `vq_loss` (see lecture slides). What is the role of the stop-gradient operator in the `vq_loss`?

b) (1.5 pts) : Complete the `TODO` sections corresponding to the visualizations under *"Part (b): VQ-VAE initialization, hyperparameters and training"*. Train the VQ-VAE model and provide a plot showing the validation loss over iterations. Provide your generated plots for visualization of the VQ-VAE model input versus reconstructions for a batch of test samples, as well as a visualization of the codebook utilization across the training set as a histogram.

c) (2 pts) : Train the VQ-VAE model with different codebook sizes $K \in \{8, 16, 32, 64\}$. Discuss the models' convergence behavior by comparing their resulting codebook utilization.

d) (2 pts) : Train the VQ-VAE model with the commitment loss weight `beta = 0.75` instead of `beta = 0.25`, and comment on the model's convergence behavior. Explain the expected impact of this change on the resulting model and the codebook utilization.

e) (1 pts) : Complete the `TODOs` under the `training_ar_prior` function of *"Part (c): Autoregressive prior over the latent space"*. Provide the code snippet corresponding to this solution. Explain how ancestral sampling from the model is performed.

f) (1 pts) : Complete the `TODO` sections under *"Part (d): Autoregressive prior initialization, hyperparameters and training"*. Describe where *"freezing the encoder of VQ-VAE"* is implemented. What would happen if we would *not* freeze the encoder at this stage?

- you can try out what happens if you don't freeze

g) (2.5 pts) : Train the autoregressive prior model and provide a plot showing the validation loss over iterations. Provide figures of generated samples from the model at different phases of training. Finally, provide visualizations of 64 generated final samples from your model. How are these final samples expected to differ from outputs obtained by independently sampling latent code indices without this learned prior?

- additional to think about -> is there a thing like "codebook collapse", i.e. it only uses a few codebook vectors to represent input